Inventory Pro

Système de Gestion d'Inventaire Professionnel

RAPPORT TECHNIQUE COMPLET

Projet de Fin d'Études (PFE)

Type : Application Web Full-Stack

Frontend : Angular 21 • Backend : ASP.NET Core

Base de données : SQL Server

Date : Mars 2026

📑 Table des Matières

1. Présentation Générale de l'Application
2. Ce que fait l'Application — Fonctionnalités Détaillées
3. Architecture Technique
4. Stack Technologique
5. Modules Fonctionnels (Frontend)
6. Architecture Backend (API REST)
7. Modèle de Données
8. Système d'Authentification & Autorisation
9. Matrice des Permissions
10. Services & Logique Métier
11. Fonctionnalités Temps Réel (SignalR)
12. APIs REST — Contrôleurs
13. Sécurité
14. Interface Utilisateur (UI/UX)
15. Déploiement & Configuration
16. Résumé des Statistiques du Projet

1. Présentation Générale de l'Application

Inventory Pro est une application web full-stack de gestion d'inventaire conçue pour permettre aux entreprises de suivre leurs produits, stocks, mouvements d'entrée/sortie, sites d'entreposage et alertes de stock en temps réel. L'application est développée dans le cadre d'un **Projet de Fin d'Études (PFE)**.

Objectifs principaux

- Gestion complète des produits (CRUD, catégorisation, code-barres, images)
- Suivi des stocks par site avec seuils d'alerte configurables

- Enregistrement et traçabilité des mouvements de stock (entrées/sorties)
- Gestion multi-sites (entrepôts, magasins, centres de distribution)
- Système d'alertes automatiques basé sur les seuils de stock
- Scanner de codes-barres intégré (caméra ou QR code externe)
- Gestion des utilisateurs avec contrôle d'accès basé sur les rôles (RBAC)
- Suggestions de réapprovisionnement basées sur les seuils
- Tableau de bord (Dashboard) avec statistiques en temps réel

2. Ce que fait l'Application — Fonctionnalités Détaillées

Cette section décrit en détail **ce que l'application permet de faire** du point de vue de l'utilisateur final, en couvrant chaque fonctionnalité majeure avec ses cas d'utilisation concrets.

📊 2.1 Tableau de Bord (Dashboard)

Le tableau de bord est la **page d'accueil** de l'application. Il offre une vue synthétique de l'état de l'inventaire en un coup d'œil :

- **Cartes de statistiques** : Nombre total de produits, valeur du stock, nombre de mouvements récents, nombre d'alertes actives — chacune avec un indicateur de tendance (hausse/baisse).
- **Top 5 des alertes actives** : Affichage instantané des alertes les plus critiques (rupture de stock, stock bas, surstock) avec code couleur par sévérité (critique, haute, moyenne, basse, info).
- **Résumé des mouvements** : Synthèse des entrées et sorties de stock récentes.
- **Statistiques des sites** : Vue d'ensemble de tous les entrepôts et magasins avec leur état actuel.
- **Produits à stock bas** : Liste des produits dont le stock est sous le seuil critique, avec statut "Critique", "Attention", ou "À commander".
- **Activités récentes** : Fil d'activité en temps réel montrant les dernières opérations effectuées.

📦 2.2 Gestion des Produits

Le module produits permet la **gestion complète du catalogue de produits** de l'entreprise :

Ce que l'utilisateur peut faire :

- **Consulter les produits** en vue **grille** (cartes visuelles avec image) ou en vue **liste** (tableau détaillé).
- **Rechercher un produit** par nom en temps réel grâce au champ de recherche instantané.
- **Filtrer par catégorie** (ex: "Électronique", "Alimentation", etc.).
- **Ajouter un nouveau produit** avec : nom, description, prix, catégorie, code-barres, et image (upload depuis le PC, stocké en Base64).
- **Scanner le code-barres** d'un produit depuis un téléphone via un QR code affiché à l'écran — le code-barres scanné est automatiquement rempli dans le formulaire grâce à SignalR.
- **Modifier un produit** existant : mettre à jour les informations, changer la catégorie, ajouter/remplacer l'image.
- **Supprimer un produit** avec confirmation.
- **Créer/supprimer des catégories** directement depuis le formulaire de produit (ajout inline).
- **Assigner un produit à des entrepôts** : Lors de la création/modification, l'utilisateur peut cocher les entrepôts où le produit doit être stocké, et des lignes de stock (quantité = 0) sont automatiquement créées.

> 🔒 **Contrôle d'accès**
>
> Seuls les utilisateurs ayant la permission `manage_products` (Admin) peuvent ajouter, modifier ou supprimer des produits. Les utilisateurs avec `view_products` (Admin, Gestionnaire, Opérateur) peuvent consulter les produits.

🔄 2.3 Mouvements de Stock (Entrées / Sorties)

Le module mouvements permet d'**enregistrer et suivre toutes les opérations d'entrée et de sortie de stock** en temps réel :

Enregistrer un mouvement :

- **Entrée de stock** — Raisons possibles : achat fournisseur, retour fournisseur, production, transfert entrant, ajustement positif, stock initial.
- **Sortie de stock** — Raisons possibles : vente, retour client, produit endommagé, périmé, transfert sortant, ajustement négatif, usage interne.
- L'utilisateur sélectionne : le **produit** (avec recherche et auto-complétion), le **site/entrepôt**, la **quantité**, la **raison**, et des **notes** optionnelles.
- Pour les sorties : sélection obligatoire d'un site de **destination**.

Validation intelligente :

- **Contrôle de stock disponible** : Lors d'une sortie, l'application vérifie que la quantité demandée ne dépasse pas le stock disponible sur le site sélectionné.
- **Contrôle du seuil maximum** : Lors d'une entrée, l'application vérifie que la quantité ajoutée ne dépasse pas la capacité restante avant le seuil maximum.
- **Filtrage des produits par stock** : En mode sortie, seuls les produits ayant du stock disponible sur le site sélectionné sont proposés.
- **Filtrage des sites** : En mode entrée, seuls les entrepôts (type "warehouse") sont affichés.

Après enregistrement :

- La **quantité de stock** est automatiquement mise à jour.
- Les **alertes** sont automatiquement recalculées (si le stock passe sous un seuil → alerte créée).
- L'utilisateur peut **scanner un code-barres avec son téléphone** (via QR code + SignalR) pour sélectionner rapidement un produit.

Consultation et filtrage :

- Historique complet de tous les mouvements avec **filtres** par : type (entrée/sortie), site, raison, date, et recherche textuelle.
- Résumé avec les **totaux** : nombre d'entrées, nombre de sorties, total des quantités.
- L'utilisateur peut consulter les détails de n'importe quel mouvement passé.
- **Ajout de raisons personnalisées** : l'utilisateur peut créer ses propres raisons d'entrée ou de sortie.

🏭 2.4 Gestion des Sites (Multi-Magasin)

L'application supporte la **gestion multi-sites** pour les entreprises ayant plusieurs entrepôts, magasins ou centres de distribution :

- **Créer un site** avec : nom, adresse, ville, code fiscal, téléphone, email, responsable, type (entrepôt, magasin, centre de distribution, production), capacité maximale, et indicateur "entrepôt principal".
- **Consulter les sites** en vue grille ou liste, avec filtrage par type et recherche.
- **Voir la capacité d'occupation** de chaque site : barre de progression colorée (vert → jaune → orange → rouge) basée sur le ratio stock actuel / capacité maximale.
- **Consulter les stocks d'un site** : Naviguer vers les stocks détaillés d'un site spécifique (`/sites/:id/stocks`).
- **Modifier ou supprimer un site** (avec confirmation).
- **Statistiques globales** : nombre total de sites, sites actifs, par type.

🔔 2.5 Système d'Alertes

Le système d'alertes **surveille automatiquement les niveaux de stock** et génère des notifications lorsque des situations anormales sont détectées :

Types d'alertes générées automatiquement :

Type	Déclencheur	Sévérité
Rupture de stock (out_of_stock)	Quantité = 0	🔴 Critique
Stock bas (low_stock)	Quantité < seuil d'alerte	🟠 Haute
Point de réapprovisionnement (reorder_point)	Quantité < seuil minimum	🟡 Moyenne
Surstock (overstock)	Quantité > seuil maximum	🔵 Basse

Ce que l'utilisateur peut faire :

- **Voir toutes les alertes** avec filtrage par : type, sévérité, date, produit, et statut (résolue ou non).
- **Consulter le détail d'une alerte** : type, message, produit concerné, site, date de création, sévérité.
- **Résoudre une alerte** : Marquer une alerte comme résolue via l'API backend.
- **Supprimer une alerte** avec confirmation.
- **Marquer toutes les alertes comme lues** en un clic.
- **Dédoublonnage automatique** : Le backend utilise un "Fingerprint" unique par type et stock pour éviter les alertes en double.
- **Statistiques d'alertes** : Nombre total, critiques, actives, par sévérité.

📱 2.6 Scanner de Codes-Barres

L'application intègre un **scanner de codes-barres multi-mode** qui permet de gérer les stocks rapidement :

Trois modes de scan :

- **Caméra intégrée** : Utilisation de la bibliothèque ZXing pour scanner via la webcam ou la caméra du périphérique. Détection automatique et en continu.
- **Saisie manuelle** : Entrée manuelle du code-barres au clavier avec validation par touche Entrée.
- **Scan téléphone distant** : L'app génère un QR code que l'utilisateur scanne avec son téléphone. Le téléphone ouvre une page dédiée (`/scan`) qui capture le code-barres via la caméra du téléphone et envoie le résultat en temps réel au PC via **SignalR**.

Actions rapides après scan :

- **Mode Consultation** : Afficher les informations du produit (nom, description, prix, catégorie, image).
- **Mode Entrée rapide** : Enregistrer une entrée de stock immédiatement après le scan (choix du site, quantité, raison).
- **Mode Sortie rapide** : Enregistrer une sortie de stock immédiatement.
- **Historique de scan** : Les 20 derniers scans sont conservés avec statut (trouvé / non trouvé).

📋 2.7 Suggestions de Réapprovisionnement

Le module de réapprovisionnement **analyse les seuils de stock** et propose des suggestions intelligentes :

- Identification automatique des produits dont le stock est **sous le seuil minimum**.
- Calcul de la **quantité recommandée** à commander pour atteindre le niveau optimal.
- Priorisation des suggestions par **urgence** (critique, attention, à commander).
- Vue par site pour savoir **quel entrepôt** a besoin de réapprovisionnement.

👥 2.8 Gestion des Utilisateurs et des Rôles

L'application fournit un système complet de **gestion des utilisateurs** réservé aux administrateurs :

Gestion des membres (Admin uniquement) :

- **Voir tous les membres** de l'équipe avec leur rôle, statut (actif/inactif), dernier login.
- **Ajouter un nouveau membre** : Nom, prénom, email, mot de passe, et attribution d'un rôle.
- **Modifier un membre** : Changer les informations, le rôle, ou le statut.
- **Activer / Désactiver un membre** : Les membres inactifs ne peuvent plus se connecter.
- **Supprimer un membre** avec confirmation.
- **Statistiques de l'équipe** : Nombre total de membres, d'admins, de gestionnaires, d'opérateurs.

Gestion des rôles et permissions :

- **Consulter les rôles** : Admin, Gestionnaire de Stock, Opérateur — chacun avec sa couleur et ses permissions.
- **Matrice de permissions** : 16 permissions organisées en catégories (consultation, gestion, opérations).
- **Attribution dynamique** : Les permissions sont vérifiées côté frontend (guards Angular) et côté backend (claims JWT).

👤 2.9 Profil Utilisateur

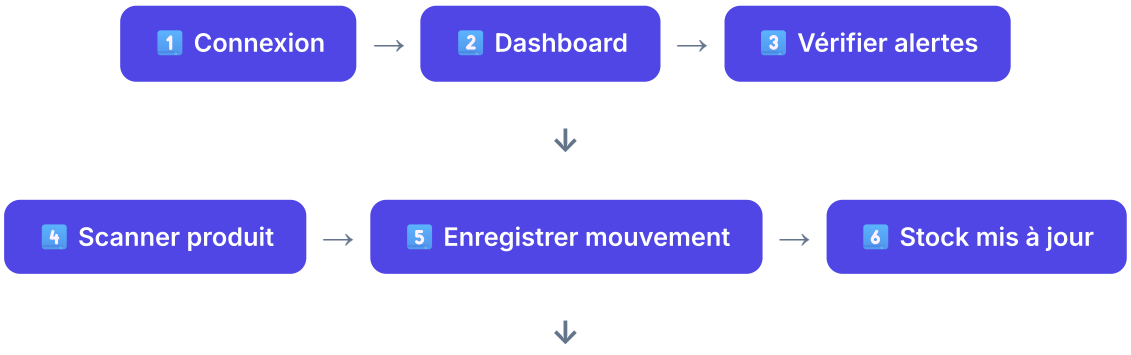
- Chaque utilisateur connecté peut **consulter son profil** : nom, prénom, email, rôle, dernière connexion.
- **Modifier ses informations personnelles**.
- Voir son **badge de rôle** avec la couleur correspondante (Rouge = Admin, Bleu = Gestionnaire, Vert = Opérateur).

⚙️ 2.10 Paramètres Système

- Accessible aux **administrateurs uniquement**.
- Configuration de l'application et des préférences système.

🔄 2.11 Flux de Travail Principal

Voici le flux de travail typique d'un opérateur ou gestionnaire utilisant l'application :



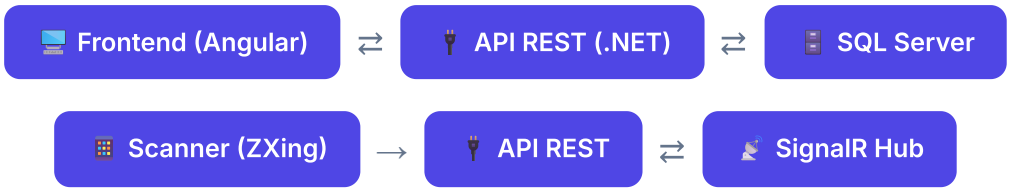
7️⃣ Alertes recalculées → 8️⃣ Réappro. suggéré → 9️⃣ Commander

🎯 En résumé, ce que fait Inventory Pro :

- ✅ Gérer le catalogue de produits avec images, codes-barres et catégories
- ✅ Suivre les stocks en temps réel sur plusieurs sites/entrepôts
- ✅ Enregistrer chaque entrée et sortie de stock avec traçabilité complète
- ✅ Alerter automatiquement quand un stock est bas, en rupture ou en surstock
- ✅ Scanner des codes-barres via caméra, téléphone distant, ou saisie manuelle
- ✅ Proposer des réapprovisionnements intelligents basés sur les seuils
- ✅ Gérer les utilisateurs avec 3 niveaux de rôles et 16 permissions
- ✅ Fournir un tableau de bord avec toutes les métriques clés
- ✅ Communiquer en temps réel entre PC et téléphone via SignalR

3. Architecture Technique

L'application suit une architecture **client-serveur** découplée avec un frontend SPA (Single Page Application) et un backend API REST.



Architecture Frontend

Pattern : Standalone Component Architecture

L'application Angular utilise des **composants autonomes** (standalone components) avec le **lazy loading** pour chaque module fonctionnel. L'état est géré par des **Angular Signals** (réactivité fine).

Architecture Backend

Pattern : Clean Architecture + CQRS

Le backend est structuré en 4 couches distinctes suivant le principe de Clean Architecture :

- **Domain** — Modèles métier, interfaces, commandes, requêtes
- **Application** — Contrôleurs API, handlers, DTOs, services, Startup
- **Data** — DbContext (EF Core), Migrations, Repositories
- **Infra** — Injection de dépendances, configuration infrastructure

4. Stack Technologique

Catégorie	Technologie	Version	Rôle
Frontend Framework	Angular	21.x	Framework SPA principal
Langage Frontend	TypeScript	5.9	Typage statique
Styling	SCSS + TailwindCSS	3.4	Styles & utilitaires CSS
UI Components	Angular Material	21.x	Composants Material Design
Scanner	ZXing (browser + library)	0.21	Lecture codes-barres/QR
QR Code	qrcode	1.5	Génération de QR codes
Temps Réel (Client)	@microsoft/signalr	10.0	Connexion WebSocket au Hub
Backend Framework	ASP.NET Core	—	API REST
ORM	Entity Framework Core	—	Accès données SQL Server
CQRS / Mediator	MediatR	—	Command/Query Separation
Mapping	AutoMapper	—	DTO ↔ Entity mapping
Auth	JWT Bearer	—	Authentification par token
API Docs	Swagger / OpenAPI	v1	Documentation interactive
Temps Réel (Serveur)	SignalR	—	Notifications temps réel
Base de données	SQL Server (Express)	—	Stockage persistant
Tests	Vitest	4.0	Tests unitaires frontend

5. Modules Fonctionnels (Frontend)

L'application est organisée en **13 modules fonctionnels**, chacun chargé dynamiquement (lazy loading) :

#	Module	Route	Description	Accès
1	**Dashboard**	`/dashboard`	Tableau de bord avec statistiques, activités récentes, top produits	Authentifié
2	**Products**	`/products`	Gestion des produits (CRUD, filtrage, vues grille/liste, images)	view_products
3	**Movements**	`/movements`	Suivi des mouvements de stock (entrée/sortie) avec historique	manage_movements
4	**Sites**	`/sites`	Gestion des sites/entrepôts (localisation, capacités, responsables)	view_sites
5	**Site Stocks**	`/sites/:id/stocks`	Stocks détaillés d'un site spécifique	view_sites
6	**Alerts**	`/alerts`	Alertes système (rupture, seuil bas, surstock), résolution	manage_alerts
7	**Réappro.**	`/reapprovisionnement`	Suggestions de réapprovisionnement basées sur les seuils	view_reappro.
8	**Scanner**	`/scanner`	Scan de codes-barres/QR via caméra avec ZXing	scan_barcode
9	**Members**	`/members`	Gestion des membres de l'équipe (rôles, statuts)	Admin only
10	**User Mgmt**	`/user-management`	Administration avancée des utilisateurs	Admin only
11	**Profile**	`/profile`	Page profil utilisateur connecté	Authentifié
12	**Settings**	`/settings`	Paramètres système	Admin only
13	**Auth/Login**	`/auth/login`	Formulaire de connexion	Public

6. Architecture Backend (API REST)

Structure des couches

```
Backend/ ├── Domain/ # Couche Domaine │ ├── Models/ # 10 entités (Product, User, Stock, Site, Alert, Role, Permission, etc.) │ ├── Commands/ # Commandes génériques (Add, Put, Remove) │ ├── Queries/ # Requêtes génériques (Get, GetList, GetPaged, Count, Exists) │ ├── Handlers/ # Handlers génériques CQRS │ ├── Interface/ # Interfaces (IGenericRepository, IJwtService, etc.) │ └── Enums/ # Énumérations ├── Application/ # Couche Application (API) │ ├── Controllers/ # 10 contrôleurs REST │ ├── Commands/ # Commandes spécifiques (StockMovement CRUD) │ ├── Queries/ # Requêtes spécifiques (GetProductByBarcode) │ ├── Handlers/ # Handlers spécifiques │ ├── Hubs/ # SignalR Hub (InventoryHub) │ ├── Services/ # Services (JWT, Alert) │ ├── Dtos/ # Data Transfer Objects │ ├── Mapping/ # AutoMapper profiles │ ├── Events/ # Événements (StockChangedEvent) │ └── Startup.cs # Configuration DI, Auth, CORS, Swagger ├── Data/ # Couche Données │ ├── Context/ # AppDbContext (EF Core) │ ├── Migrations/ # Migrations SQL Server │ └── Repositories/ # GenericRepository<T> └── Infra/ # Couche Infrastructure └── DependencyContainer.cs
```

7. Modèle de Données

La base de données **SQL Server** (nom : `Inventaire`) contient les entités suivantes :

Entité	Champs Principaux	Relations
User	Id_u, Nom, Prenom, Email, MotDePasse, Status, RoleId, LastLogin	→ Role (N:1), → StockMovement (1:N)
Role	RoleId, Nom, Description	→ User (1:N), → RolePermission (1:N)
Permission	permissionId, Code_p, Description	→ RolePermission (1:N)
RolePermission	RoleId, PermissionId	→ Role (N:1), → Permission (N:1)
Product	id_p, Nom, Description, CodeBarre, Prix, id_c, ImageUrl	→ Category (N:1), → Stock (1:N)
Category	id_c, Nom, Description	→ Product (1:N)
Site	Id_site, Nom, Adresse, Ville, code_fiscale, Telephone, Email, ResponsableSite, Type, Capacite, EstEntrepotPrincipal	→ Stock (1:N)
Stock	id_s, QuantiteDisponible, SeuilAlerte, SeuilSecurite, SeuilMinimum, SeuilMaximum, id_p, Id_site	→ Product (N:1), → Site (N:1), → StockMovement (1:N), → Alert (1:N)
StockMovement	id_sm, DateMouvement, Raison, Quantite, Type (entry/exit), Note, Destination, Id_s, Id_u	→ Stock (N:1), → User (N:1)
Alert	Id_a, Type, Message, DateCreation, Resolue, Severity, Status, Fingerprint, ClosedAt, id_s	→ Stock (N:1)

Diagramme Relationnel (Résumé)

```
User ──N:1──▶ Role ◀──1:N── RolePermission ──N:1──▶ Permission Product ──N:1──▶ Category
Product ──1:N──▶ Stock ──N:1──▶ Site Stock ──1:N──▶ StockMovement ──N:1──▶ User Stock ──1:N──▶
Alert
```

8. Système d'Authentification & Autorisation

Authentification (JWT)

- Le backend émet des **tokens JWT** signés avec HMAC-SHA256
- Durée de validité : **1440 minutes** (24 heures)
- Issuer : `InventaireApi` | Audience : `InventaireApp`
- Le token est stocké côté client et envoyé via le header `Authorization: Bearer <token>`

Flux d'authentification

Identifiants → API /auth/login → Validation → JWT Token → Dashboard

Autorisation (RBAC)

L'application implémente un système **RBAC (Role-Based Access Control)** avec 3 rôles prédéfinis :

Rôle	Nom Backend	Couleur	Description
Admin	`admin`	Rouge (#f44336)	Accès complet au système, gestion des utilisateurs et rôles
Gestionnaire de Stock	`gestionnaire_de_stock`	Bleu (#2196f3)	Gestion des mouvements, alertes, rapports. Pas d'accès à la gestion des utilisateurs
Opérateur	`operateur`	Vert (#4caf50)	Opérations de base : scan, consultation produits, entrée/sortie

Protection des Routes (Frontend)

Les guards Angular protègent les routes :

- `authGuard` — Vérifie l'authentification
- `adminGuard` — Restreint aux administrateurs
- `stockManagerGuard` — Restreint aux gestionnaires de stock
- `permissionGuard('permission')` — Vérifie une permission spécifique

9. Matrice des Permissions

Le système définit **16 permissions** réparties en 3 catégories :

Permission	Description	Admin	Gestionnaire	Opérateur
📋 **Permissions de Consultation**				
`view_dashboard`	Voir le tableau de bord	✓	✓	✗
`view_products`	Voir les produits	✓	✓	✓
`view_movements`	Voir les mouvements	✓	✓	✗
`view_sites`	Voir les sites	✓	✓	✗
`view_alerts`	Voir les alertes	✓	✓	✗
`view_reports`	Voir les rapports	✓	✓	✗
`view_reapprovisionnement`	Voir le réappro.	✓	✓	✓
⚙️ **Permissions de Gestion**				
`manage_products`	Gérer les produits	✓	✗	✗
`manage_movements`	Gérer les mouvements	✓	✓	✗
`manage_sites`	Gérer les sites	✓	✗	✗
`manage_alerts`	Gérer les alertes	✓	✓	✗
`manage_users`	Gérer les utilisateurs	✓	✗	✗
`manage_roles`	Gérer les rôles	✓	✗	✗
`manage_reapprovisionnement`	Gérer le réappro.	✓	✓	✗
🔧 **Permissions Opérationnelles**				
`scan_barcode`	Scanner les codes-barres	✓	✗	✓
`basic_entry_exit`	Entrée/sortie basique	✓	✗	✓

10. Services & Logique Métier

Services Frontend (Angular)

Service	Fichier	Responsabilité
AuthService	auth.service.ts	Authentification, gestion JWT, session utilisateur
AuthorizationService	auth-authorization.service.ts	Vérification permissions, contrôle d'accès RBAC
ProductService	product.service.ts	CRUD produits, filtrage, recherche
MovementService	movement.service.ts	Gestion mouvements de stock (entrée/sortie)
SiteService	site.service.ts	CRUD sites/entrepôts
StockService	stock.service.ts	Gestion des niveaux de stock par site
AlertService	alert.service.ts	Gestion alertes, résolution, marquage lu
CategoryService	category.service.ts	Gestion des catégories de produits
RolesService	roles.service.ts	Gestion des rôles et permissions backend
UserService	user.service.ts	CRUD utilisateurs
ScanSessionService	scan-session.service.ts	Gestion de sessions de scan codes-barres

Services Backend (.NET)

Service	Responsabilité
JwtService	Génération et validation des tokens JWT
AlertService	Moteur d'alertes : génération automatique basée sur les seuils de stock, dédoublonnage par Fingerprint
GenericRepository<T>	Repository générique EF Core pour toutes les entités (CRUD, pagination, comptage)

Patterns CQRS (MediatR)

Handlers Génériques

Pour chaque entité (Product, User, Stock, Site, Alert, Category, Role, StockMovement), les handlers suivants sont enregistrés :

- `AddGenericHandler<T>` — Création
- `PutGenericHandler<T>` — Mise à jour

- `GetGenericHandler<T>` — Lecture unitaire
- `GetListGenericHandler<T>` — Liste complète
- `GetPagedListGenericHandler<T>` — Liste paginée
- `RemoveGenericHandler<T>` — Suppression
- `GetCountGenericHandler<T>` — Comptage
- `ExistsGenericHandler<T>` — Vérification d'existence

11. Fonctionnalités Temps Réel (SignalR)

L'application utilise **SignalR** pour les notifications temps réel :

- **Hub Endpoint** : `/hubs/inventory`
- **Événements** : `StockChangedEvent` déclenche le handler `StockChangedEventHandler`
- **Usage** : Notification automatique quand un mouvement de stock modifie la quantité disponible, déclenchant potentiellement une alerte

12. APIs REST — Contrôleurs

Contrôleur	Base URL	Opérations
Authentification	/api/auth	Login, Register, Token refresh
ProductsController	/api/products	CRUD produits, recherche par code-barre
CategoriesController	/api/categories	CRUD catégories
StocksController	/api/stocks	CRUD stocks, gestion seuils, stock par site
StockMovementsController	/api/stockmovements	Créer/modifier/supprimer mouvements, historique
SitesController	/api/sites	CRUD sites/entrepôts
AlertsController	/api/alerts	CRUD alertes, résolution, marquage
UsersController	/api/users	CRUD utilisateurs, changement de rôle, activation/désactivation
RolesController	/api/roles	CRUD rôles, affectation permissions
ProfileController	/api/profile	Consultation/modification profil connecté

13. Sécurité

Mesure	Détails
Authentification JWT	Tokens signés HMAC-SHA256 avec validation Issuer/Audience/Lifetime
RBAC	Contrôle d'accès basé sur les rôles côté frontend (guards) et backend (claims JWT)
CORS	Configuré pour autoriser le frontend Angular
Protection XSS	Sanitization automatique Angular
Guards de routes	4 types de guards : authGuard, adminGuard, stockManagerGuard, permissionGuard
Swagger sécurisé	Documentation API avec schéma Bearer Token
Dédoublonnage alertes	Fingerprint unique par type + stock pour éviter les doublons

14. Interface Utilisateur (UI/UX)

Design System

- **Framework UI** : Angular Material + composants personnalisés SCSS
- **Utilitaires CSS** : TailwindCSS 3.4 pour les classes utilitaires
- **Icônes** : Material Icons
- **Typographie** : Police Inter (Google Fonts)
- **Palette** : Gradient principal violet (#667eea → #764ba2)

Éléments d'interface

- **Sidebar rétractable** avec icônes, badges de notification et navigation contextuelle par rôle
- **Header** avec barre de recherche globale, notifications et menu utilisateur
- **Cartes statistiques** avec tendances et indicateurs visuels
- **Vues grille/liste** pour les produits avec filtrage avancé
- **Badges de rôle** : composant `<app-role-badge>` coloré par rôle
- **Design responsive** : Desktop, Tablette et Mobile avec menu hamburger
- **Animations** : Transitions et hover effects via Angular Animations

14. Déploiement & Configuration

Prérequis

- Node.js v18+ | npm v9+
- SQL Server Express
- SDK .NET

Configuration Frontend

```
# Installation des dépendances npm install # Démarrage du serveur de développement npm start #
URL : http://localhost:4200
```

Configuration Backend

```
# Chaîne de connexion (appsettings.json)
Server=MSI\SQLEXPRESS;Database=Inventaire;Trusted_Connection=True # Swagger UI http://localhost:
<port>/swagger/index.html
```

Proxy Angular → Backend

Le fichier `proxy.conf.json` redirige les appels API du frontend vers le backend .NET.

15. Résumé des Statistiques du Projet

13
Modules Frontend

10
API Controllers

10
Entités de données

11
Services Frontend

16
Permissions

3
Rôles utilisateurs

4
Guards de routes

4
Couches Backend

Technologies Utilisées (Résumé)

Angular 21
Frontend SPA

.NET Core
Backend API

SQL Server
Base de données

SignalR
Temps réel